Detecting Prediction Collapse Caused by Pooled Feature Normalization in Space-Trajectory Screening

HARSHA VARDAN M SAKAMURI
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai 600127, India

ROHIT MICHAEL RAJ
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai 600127, India

VALARMATHI P.
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai 600127, India

Abstract—A single feature scaler is often used among a set of data. We show that it can quietly harm a sequence model while the particular scale-invariant XGBoost baseline tested here remains healthy and a pooled validation score shows the damage only in part. A Transformer trained with a scaler averaged over a target group ceases to react to the input on Venus (3.11e-05 prediction standard deviation on unseen missions, 0.6037 validation area under the ROC curve (AUC)). The same transformed arrays remain almost perfectly separable by XGBoost, whose AUC varies by at most 0.0002 across the three normalization settings. This is a narrow invariance result, not a claim that tree models generally cannot diagnose preprocessing errors. The failure is selective: 1 of 4 targets collapses; the remaining targets lose between 0.0003 and 0.0925 AUC while continuing to discriminate. Collapse occurs on 4 of 5 seeds, so five seeds do not establish universal reliability. Our AI contribution is a controlled test of when pooled normalization breaks a sequence model, and a prediction-spread statistic that needs no labels at inference and exposes the failure, with its threshold set on validation runs. On NASA turbofan data the signal also shrinks about nine times, but the model keeps working. The engineering application is screening simulated spacecraft trajectories to save compute; operational integration is proposed, not flight-demonstrated.

Manuscript received XXXXX 00, 0000; revised XXXXX 00, 0000; accepted XXXXX 00, 0000.
This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. (Corresponding author: Valarmathi P.)

Valarmathi P. is with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai 600127, India (e-mail: valarmathi.sudhakar@vit.ac.in). Harsha Vardan M Sakamuri and Rohit Michael Raj are with the same school.

The code, the result files and the source of this article are at https://github.com/Haise-727/gmat-pred.

0018-9251 © 2026 IEEE

Index Terms—Feature normalization, model diagnostics, prediction spread, shortcut learning, simulation screening, trajectory analysis

I. INTRODUCTION

In Monte Carlo trajectory campaigns, the majority of the compute is used on runs that were already sure to fail when they left the parking orbit. These campaigns are a regular occurrence at flight facilities, such as the large dispersion campaign that TESS used to set its trajectory design [1]. Not only is this expensive, a body of work has been developed to make each propagation even less expensive [2], [3]. Another, and obvious, approach is to steer clear of these runs: track the first part of a trajectory and cancel out the ones that will fail. We built such a screen. Building it exposed a failure that we expect to apply beyond astrodynamics.

This paper deals with that failure. A single feature scaler was fitted to grouped data and used to train the model, and the model then produced a single constant output per group. That a pooled scaler can do this is not surprising. The difficulty is that it is easy to miss. A pooled validation score shows the damage only in part: in the controlled ablation below, the AUC over the three targets of a group falls from 0.9995 to 0.875, while the AUC of the failed target alone is 0.687. A gradient boosted baseline on the same transformed arrays gave almost perfect results, because trees split on the values of the features and the rescaling on which the network depends does not matter to them.

Running a simple baseline is standard advice, and the result above is in tension with it. It is common practice for tabular and physical data to be checked by tree ensembles and they are often a strong choice [4], [5], [6]. In our controlled comparison, the particular XGBoost baseline is invariant to the monotone rescaling that damages the network, so its success does not test whether this network can use the compressed signal. Other tree-based checks, other feature audits, or trees exposed to a non-monotone or information-destroying preprocessing error could diagnose other failures; our evidence does not say otherwise.

The failure can be described as a preprocessing error, and its remedy, scaling each group separately, is simple. The contribution of this paper is not the remedy. It is the characterization of a failure that a pooled validation score shows only in part and that the tested scale-invariant baseline does not show, together with a check on the spread of the predictions that does show it.

We make three contributions. It is first a controlled decomposition of the collapse, which distinguishes pooling over time from pooling across groups, and shows only the latter to be deadly. Second, a demonstration that a scale-invariant baseline actually

approves of the broken configuration, as well as an inexpensive diagnostic, the spread of the predictions, that does notice it. Its threshold is set on validation predictions and applied to test predictions, and Section VI states which choices were made after the results were seen. Third, a second dataset, NASA turbofan engines, where the signal shrinks in the same way and the model keeps working, which shows that the shrinking alone is not the problem.

This paper is based on scripts in the attached repository that generate all the numbers, which are written to machine-readable artifacts. Those artifacts are then used to create the paper containing tables and figures.

II. RELATED WORK

Normalization is crucial for training deep networks, and there are standard layers (batch, layer, group, instance) that solve a version of the problem [7], [8], [9], [10], [11]. It has been long known that the ability of a network to train, and not merely the rate at which it trains, depends on the scaling of the inputs [12]. These layers also make some assumptions about the input that they are given. They assume that the feature statistics are stationary, or that they are comparable within each pooling axis. When more than one set of data is measured on the same scaler, but the range of the data is very different, that assumption is already broken before the first layer runs. Normalization has also been studied for time series on its own. ST-Norm splits it into a temporal part and a spatial part for forecasting models [13], and Deep Adaptive Input Normalization learns the scaling from the data [14]. Those papers want a more accurate model. We ask a different question: how can a bad scaling choice break a model without anyone noticing? What we add is a controlled decomposition that shows us that it is not the pooling in general that does the damage, but only the pooling across the groups, and then an observation that the resulting model is not a weak model, but a constant model.

When a shortcut is there, a network will easily learn the easy signals and take the shortcut over the intended structure [15], [16]. Previous research on Clever-Hans behavior demonstrates that the models achieve good scores for reasons that are not related to the task [17]. If the scales are grouped and are not homogenous, there is an exceptionally clean shortcut, the distance between the group centroids. This type of model can memorize the difference between groups, and can reach a high AUC on a mixed validation set while performing no better inside individual groups. In our ablation the pooled AUC of the inner group falls to 0.875, while the AUC of Venus alone is 0.687 (Section VI). Here, our contribution is a diagnostic (spread of the predictions within a group) which does not need any label and is quite different from the common underfitting. It is also not a modeling problem, but a pipeline level problem, like many which can be found in the shared preprocessing code that no single model owns [18].

Tree ensembles perform well on tabular and physical data, and often perform with the same or better results than deep models [4], [5], [6], [19], [20], and many practitioners turn to a tree as a baseline for that reason. Their splits are axis-aligned and threshold-based, so a monotone rescaling can preserve the available ordering. That useful invariance is why the tested XGBoost baseline misses this particular cross-group scaling failure when it receives the same transformed arrays; it does not imply that tree baselines are generally unable to identify preprocessing errors.

The word ‘collapse’ also needs disambiguation. Neural collapse conventionally describes the terminal-phase geometry of within-class representations and classifiers after training error vanishes [21]; our outcome is different: predictions over distinct missions become nearly constant after cross-group scaling. Work on predictive uncertainty under shift shows that confidence and calibration can deteriorate when the deployment distribution changes [22], while two-sample tests can detect changes between reference and incoming feature distributions without target labels [23]. Production-readiness rubrics likewise call for data, model, and serving-pipeline monitors rather than relying on a single held-out score [24]. Table I makes these boundaries explicit.

Networks have been successfully applied in astrodynamics for trajectory optimization, approximation of optimal control [25], [26], [27], cost estimation without complete propagation [28], [29] and learning of a robust low thrust policy with certification via Monte Carlo [30]. Learned models also correct orbit predictions from sparse tracking [31], stand in for Monte Carlo when propagating orbit uncertainty forward [32], and are used for anomaly and fault detection in spacecraft telemetry [33], [34], where the same family of attention architectures is used as here [35]. Screening specifically, by means of a learned model selecting which of a very large number of candidate cases get full analysis, is already recognized: an ESA competition on real conjunction data [36], [37], and a benchmark of deep approaches against the classical filters they would replace [38]. The final one is the most similar setting, a screening task on a deterministic astrodynamics simulator, measured against non-deep baselines. We could not find an article in this literature that considers early termination of individual Monte Carlo trajectory runs as we do in Section VIII. The closest work decreases the cost of each propagation rather than deciding which propagations to start [2], [3]. The general idea is to use a learned surrogate instead of part of an expensive simulation [39].

TABLE I

Comparison with adjacent diagnostics. Label-free inference does not remove the need for development-time threshold calibration and evaluation.

Prior method	Failure detected	Label requirement	Dependence on a reference model	Limitation addressed by our approach
Neural-collapse geometry [21]	Terminal class-feature geometry	Class labels; internal activations	No external model	Targets output constancy from grouped preprocessing, not terminal-phase geometry.
Predictive uncertainty [22]	Uncertainty and calibration under shift	Labels for evaluation	Method-dependent; often ensembles	Directly tests a distinct output degeneracy.
Dataset-shift tests [23]	Source–target distribution change	No target labels for alarm	Reference distribution; sometimes a domain classifier	Tests within-group model outputs, not only input shift.
ML pipeline validation [24]	Broad data, training, serving regressions	Test-dependent	Schema, baseline, or prior version	Supplies a concrete group-level output invariant.
This work	Near-constant within-group predictions	None to compute; labels to calibrate/evaluate	None for spread; optional XGBoost triage	Isolates cross-target pooling and separates collapse from low AUC.

III. SETUP

In total, we developed 70,000 missions that leave the Earth, 10,000 missions to each of seven interplanetary targets (Mercury, Venus, Mars, Jupiter, Saturn, Uranus and Neptune), using NASA's General Mission Analysis Tool. It has an extensively verified two and three body propagator [40], and is deterministic. Each mission has six injection offsets, which are the components of the trans orbit insertion burn (dv_V, dv_N, dv_B), and the RAAN, AOP and INC of the parking orbit from which the mission departs. Each is classified as success or failure, and failed ones are further sub-categorized as surface impact, orbit too high, missed target and a few others.

The interplanetary transfers are sampled at a fixed rate of 54,000 s (15 hours) and downsampled to about 100 steps for each mission. This is so that the sequences produced by targets with flight times that vary by 2 orders of magnitude are comparable. Thirteen features are recorded for each step, and include position in the synodic frame, specific orbital energy, flight path angle, normalized target distance and eccentricity. An eighth target was created (the Moon), and is intentionally left out of the study. It is a 6 day transfer inside Earth's neighborhood, sampled at 60 s, and it does not have a cost structure similar to Section VIII, or a heliocentric dynamical regime.

The screening task is to observe the first 40% of the mission's flight path and decide whether or not to abort. Splits are mission based 70/15/15, deterministic in (n, seed). The experiments are all based on a single definition in the repository, so no model is ever scored using data that was used for model selection. It is not a theoretical question. Two of the previous analyses in this project shuffled the same number of missions using the same seed, and in those analyses the same set of missions ended up in both 'test' sets.

The sequence model is a Pre-LN Transformer encoder, d_model 128, 8 heads, 4 layers and CLS token pooling, and 2 heads are connected to a single trunk (one for mission outcome and one for failure mode). It is trained on random length prefixes, so that its predictions are not only at the 40% mark in the stream but at any point in the stream. The baseline is XGBoost, 300 trees at depth 5, fitted to the same normalized prefix, flattened. It is important to note that the baseline is not working from a feature set derived separately, but from the same view the network sees, which is what 'the same input' means throughout this paper. That detail carries most of the argument so we keep it fixed.

The three conditions are the following sets of normalizations, which only vary in how the feature statistics are combined. All use the same architecture, seed, split, optimizer and epoch budget.

per timestep: every one takes its statistics from the distribution of features at that timestep index, over the missions of one target (Fig. 1). This is the configuration the working system uses.

global: one RobustScaler (median and IQR) per target, pooled across that target's own timesteps.

grouped: one RobustScaler pooled across all timesteps of all targets in a regime group (inner: Mercury, Venus, Mars; outer: Jupiter, Saturn, Uranus, Neptune). This is the setting that was not successful in production.

The third setting is not an artificial construction. A single RobustScaler fitted to a concatenated training set is what many standard preprocessing tutorials produce. It raises no warning, and the result looks like an ordinary normalized array.

IV. GROUPED-NORMALIZATION COLLAPSE

A mission's features sweep a wide range as they fly, and the range other targets occupy varies by orders of magnitude. A scaler applied to a set of pooled data will then scale the data on the scale of the largest source of variation in the set. What the screen should be able to tell apart is neither of those things. It is the spread between missions at a particular point in the flight, which by comparison is small. Pooling divides that spread by an arbitrary denominator, which drives

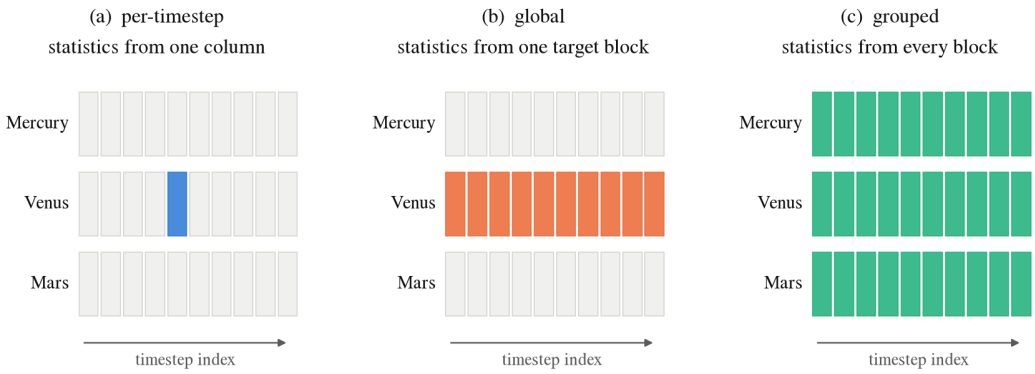


Fig. 1. The three normalization modes, and the only difference between them: from what part of the data does the scaler get its median and IQR? Everything else in training is the same. Each cell is all the missions of one target at one timestep index, and the shaded cells are the ones the median and IQR are estimated from.

the level of the useful signal toward zero and has no effect on the structure between groups. The network is then tasked with learning a difference that is close to the limit of what gradient descent can detect, and Pre-LN layer normalization filters out a lot of it.

We use the signal ratio, defined as the average standard deviation of the normalized features across missions, calculated only in the observed window. The window matters. Averaged over the entire flight the metric is uninformative: late timesteps have a large spread across missions, because the missions that fail have physically diverged by then, and this spread swamps the early signal, so that all conditions appear alike. The model commits at 40%, so the relevant range is the one inside the prefix that the model observes.

The first result is a controlled decomposition. Pooling across time within a single target reduces the signal by up to a factor of 23, and the network still works on all 4 targets, with a loss of a few thousandths of AUC at most. In contrast, pooling feature statistics across heterogeneous target groups produces substantially more severe degradation, including near-constant predictions for Venus: the output standard deviation is 3.11e-05, which is one value for every mission, and the AUC drops from 0.9999 to 0.6037.

We use the term ‘collapse’ and not ‘underfitting’ because of Fig. 2. A low AUC alone is ambiguous, since it is also produced by a model that has learned something poor or wrong. A predicted distribution on unseen missions that is three ten-thousandths wide is not ambiguous: the output no longer depends on the input.

The second result is that the collapse is selective, and we report that alongside the case that breaks. In the remaining 3 targets the same manipulation leaves the output still varying, and costs between 0.0003 and 0.0925 AUC. The signal ratio is not a measure of severity either. Jupiter compresses its signal to 0.0247, whereas Mercury compresses its signal to 0.0484, and yet Jupiter loses 0.0003 AUC while Mercury loses 0.0925 AUC. One explanation is that the Jupiter task is easily separable, so that even a very weak signal is sufficient. We did not test this directly. Compression is necessary for the collapse but not sufficient; how hard the underlying task is modulates it. We are not claiming that these data support a rule of the form ‘it collapses when the ratio is below X’. The counter example is visible in Fig. 3.

One note on scope. This ablation isolates normalization and no other component of the model, so only the statistics are pooled, with one model trained for each target. The production failure also shared a single trunk across the group, which compounds the problem, since one trunk then has to serve targets whose inputs have all been flattened in the same direction, toward a constant. These numbers therefore represent a minimum damage level of the configuration. They show that pooled normalization is sufficient on its own to destroy a target, and not that this is a complete reconstruction of the original incident.

Under the grouped setting the tested XGBoost configuration reports AUC 1.0000 on Venus, the target where the network has collapsed to a constant at AUC 0.6037. Its AUC changes by at most 0.0002 across settings on all targets, which implies that this normalization choice is not visible to this baseline. It therefore establishes that discriminative ordering remains in the transformed arrays, but it does not establish that the deep model can exploit the remaining magnitude of variation.

The exact claim matters here. The information is there and this tree can use it. This particular baseline comparison is not informative on the question that has to be answered, which is ‘can the network use the compressed signal?’. Its invariance to this rescaling makes this check orthogonal to this defect. That

statement is deliberately narrower than claiming that tree baselines cannot diagnose preprocessing errors in general.

A pooled validation score shows the collapse only in part. For the inner group (Mercury, Venus and Mars) the AUC over all three targets falls from 0.9995 to 0.875 under grouped normalization, while the AUC of Venus alone is 0.687, because a model that is constant within each group still ranks the groups against each other. The pooled value is higher than the value for the worst target and does not say which target failed. Two checks identify it.

Prediction spread within a group. For group g with a batch of n_g predictions, we compute $s_g = \mathrm{sd}\{P(\text{fail} \mid x_i) : i \in g\}$. This statistic is label-free at inference: it uses predictions and group membership, not outcomes. Labels are nevertheless needed during development to decide which low-spread states are harmful, calibrate an alert level, and estimate false alarms and missed detections.

Tested XGBoost against the network under the same preprocessing. If XGBoost has a big lead with both fed the same array, then usable ordering remains for that model while the network has failed to exploit the compressed signal.

Both are inexpensive and neither necessitates that one predicts this exact defect beforehand. In the first experiments, 'collapsed' was assigned when $s_g < 10^{-3}$. This rule was written into the experiment before the multi-seed replication, but after the original single run, so it is a heuristic definition and not a threshold fitted to labeled validation cases. Test labels and test AUC were not used to choose it. Labels from the same rule cannot also serve as ground truth for sensitivity or specificity, because that would be circular. Section VI therefore uses a definition based on AUC, and checks a threshold on validation and test predictions. The evidence supports the statistic as a conspicuous signature in this experiment, not 10^{-3} as a universal operating threshold.

A. Mission-assurance interpretation and operational use

The first paragraph below states what the experiments in this paper show. The second and third describe consequences and a workflow that have not been tested.

Demonstrated result. Within the GMAT experiment, the group-level output statistic exposes four near-constant Venus runs that the tested XGBoost comparison does not identify and that a pooled AUC shows only in part. Because it checks the behavior at the preprocessing–model interface, the same form of invariant can supplement verification and validation in other simulation-screening pipelines: it tests whether changing candidate inputs still changes the learned screen. It does not validate the trajectory propagator, prove physical correctness, certify flight software, or establish a threshold outside the evaluated configuration.

Operational implication. If an analogous collapse remained undetected in a mission-design campaign, the screen could assign nearly the same disposition to materially different candidates. Depending on that constant and the decision threshold, it could discard viable trajectories, retain expensive failures, distort estimates of feasible-region size, and bias trade studies or robustness margins. A healthy aggregate score from other target groups could conceal the affected group and create false confidence in the campaign. These consequences are failure analyses, not outcomes measured in a flight campaign.

Proposed pipeline. During development, the team would generate labeled working, degraded, and intentionally fault-injected validation states for each dataset–architecture–target-group–preprocessing version. It would calibrate a group-specific spread threshold and a minimum batch size on those validation states, freeze both before the untouched test evaluation, and record false-alarm and miss rates with uncertainty. At runtime, each screening batch would log model, scaler, data, and group versions; compute s_g alongside robust spread, input-signal, and feature-shift checks; and compare only with the matching frozen limits. An alarm would quarantine the learned screen, preserve all candidates, and fall back to the full-fidelity propagation rather than aborting trajectories. An XGBoost comparison can aid offline triage by asking whether information remains in the same arrays, but is not required to compute the inference-time statistic. Once delayed outcomes are available, they would support periodic re-evaluation and versioned recalibration, and never a silent transfer of a threshold from another domain.

V. DOES IT REPRODUCE ACROSS SEEDS?

Is it spread by seed? All of the above is problematic in that it is one run deep. A bad initialization could result in an AUC of only 0.6037. Thus, we redo the whole ablation in 5 different seeds, in each of which we redraw the train/validation/test partition and the initialization.

Compression is not dependent on seed. The signal ratio for Venus under grouped normalization lands between 0.00222 and 0.00232 across all 5 runs, a spread of 5% against the compression of 335x itself. The healthy condition is stable too. The difference in per timestep AUC between two different seeds is at most 0.0010. There is no seed on which Mars, Mercury or Jupiter collapse.

It depends on the seed whether the network escapes or not. Venus meets the pre-specified experimental rule $s_g < 10^{-3}$ on 4 of 5 seeds, with prediction spread on unseen missions ranging from 1.57e-05 to

TABLE II

Normalization ablation. Identical architecture, seed and split; only the pooling differs. ‘sig’ is the mean within timestep standard deviation of the normalized features in the observed window. ‘std’ is the standard deviation of P(fail) on unseen missions, the quantity that identifies a collapse, as against a model that is merely inaccurate.

	per-timestep			global			grouped		
Target	sig	AUC	std	sig	AUC	std	sig	AUC	std
Venus	0.7681	0.9999	4.72e-01	0.0518	0.9947	4.50e-01	0.0023	0.6037	3.11e-05
Mars	1.0829	0.9994	4.33e-01	0.0472	0.9972	4.00e-01	0.8005	0.9422	4.06e-01
Mercury	0.7686	0.9994	4.27e-01	0.1321	0.9970	4.23e-01	0.0484	0.9069	4.25e-01
Jupiter	0.7708	1.0000	4.78e-01	0.0763	1.0000	4.78e-01	0.0247	0.9997	4.75e-01

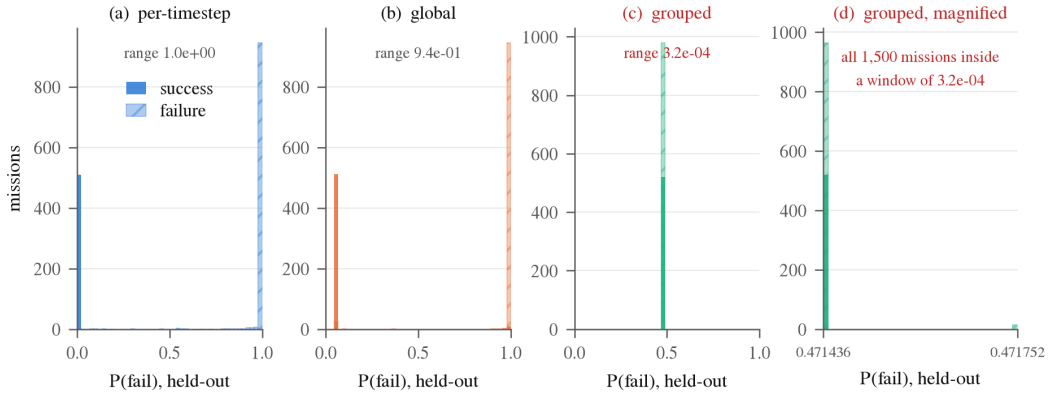


Fig. 2. P(fail) on Venus, split out for each setting. Per timestep and global normalization put the two populations at opposite ends of the axis. Under grouped normalization every mission gets essentially the same number. Panel (d) magnifies that spike and shows the full range covers about three ten-thousandths. This is what makes a collapse different from a merely bad model.

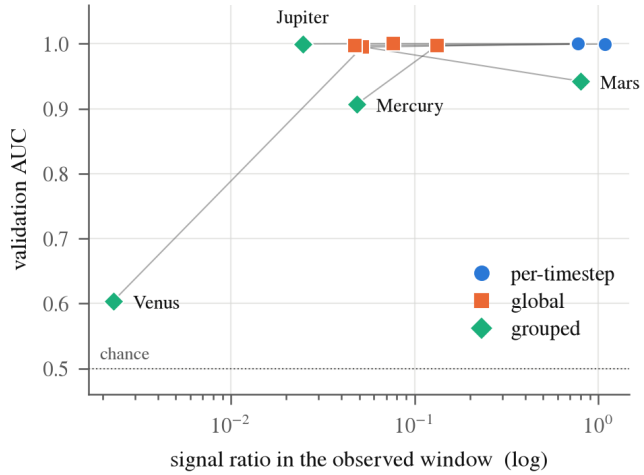


Fig. 3. Signal ratio against validation AUC. These points would lie on a curve if the collapse was only due to compression. Jupiter has a lower signal ratio than Mercury and still loses very little.

4.84e-05, which is 4 orders of magnitude lower than the 4.66e-01 obtained by correct normalization. On the remaining seed it does not meet the collapse rule, and it does not recover either: prediction spread 3.64e-02, still an order of magnitude short, at AUC 0.9305 against a healthy 0.9999. The observed recurrence is 4/5, with a 95% Wilson interval of 0.376 to 0.964. That wide interval describes recurrence of the defined state under these seeds; it is not a confidence interval for detector sensitivity and does not support universal reliability.

TABLE III

The tested XGBoost model on the same normalized window for each setting. Its AUC is nearly invariant to the particular rescaling that destroys the network.

Target	per-timestep	global	grouped	spread
Venus	1.0000	1.0000	1.0000	0.0000
Mars	0.9996	0.9998	0.9998	0.0002
Mercury	0.9997	0.9998	0.9998	0.0001
Jupiter	1.0000	1.0000	1.0000	0.0000

This is the real reason for the prediction spread diagnostic above. The range of grouped AUC for Venus over these runs is 0.5338 to 0.9305, which overlaps that of a merely weakened target. Mercury sits inside that range on every seed while still telling missions apart. AUC is thus unable to distinguish the two states. Prediction spread can. There are orders of magnitude between the collapsed runs and all the healthy runs. If you have to keep tabs on one number for this failure, watch that one.

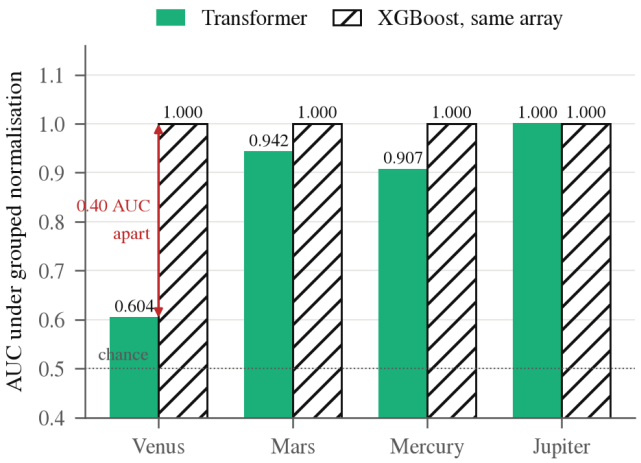


Fig. 4. Both models under grouped normalization on the same arrays. The tested XGBoost model is at the ceiling on every target, including the one where the network is just above chance.

TABLE IV

Validation AUC across 5 seeds, mean and standard deviation, and the number of seeds on which the grouped run collapsed to a constant output.

Target	per-timestep AUC	grouped AUC	collapsed
Venus	0.9998 ± 0.0001	0.6912 ± 0.1530	4/5
Mars	0.9992 ± 0.0004	0.9457 ± 0.0093	0/5
Mercury	0.9997 ± 0.0002	0.9118 ± 0.0036	0/5
Jupiter	0.9999 ± 0.0002	0.9997 ± 0.0003	0/5

The same invariance appears across the replication. Across 5 seeds, 4 targets and three normalization settings, which is 60 fitted XGBoost models in total, AUC stays within 0.9989 to 1.0000, and within each seed and target the AUC difference is at most 0.0003 between settings. Thus this tested baseline does not distinguish these normalization conditions. This is evidence about one baseline–failure pairing, not all tree-based diagnostics.

VI. A CHECK THAT FINDS THE COLLAPSE

A collapsed model gives nearly the same P(fail) to every mission. So we look at how much its predictions vary. The score is twice the group spread s_g of Section IV, so that it runs from 0 to 1:

$$S = 2\,\mathrm{Std}(\hat{p}), \tag{1}$$

where $\hat{p}$ is the predicted P(fail) on the held out missions of one target. It is 0 for a constant output, and 1 for the widest spread a probability can have. It needs no labels at inference time and no second model. Labels are used only in development, to say which runs have collapsed and to set the threshold. The four runs that this definition marks as collapsed are the same four that fall below the earlier rule $s_g < 10^{-3}$.

The second score is the one a reviewer would try first. It divides by the spread of a healthy model on the same data:

$$D = \frac{\mathrm{Std}(\hat{p})}{\mathrm{Std}(\hat{p}_{\mathrm{healthy}}) + \epsilon}. \tag{2}$$

It needs a model that was trained the right way, and when something is broken that is the thing we do not have. So we use S as the main score and report D to compare.

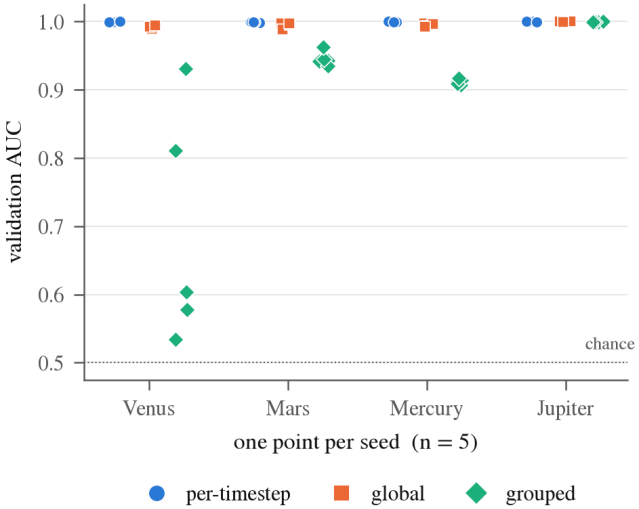


Fig. 5. Every seed plotted on its own rather than summarized as an interval. With n = 5 the spread of the raw points is the honest summary.

To get models in every state we repeat the ablation of Section IV on 5 seeds, 4 targets and 3 settings, which is 60 runs, and keep the predictions on the validation and the test missions of each run. The settings give four states. Per timestep runs are healthy. Global runs are mildly hurt. Grouped runs that keep working are severely hurt. Grouped runs that cannot rank missions are collapsed. A run counts as collapsed if its AUC on that split is 0.85 or lower.

The choices in this evaluation were made in the following order. The group definitions and the three normalization settings come from the ablation of Section IV. The AUC cutoff of 0.85 that defines a collapsed run was set after the validation and the test AUCs of all 60 runs had been seen. It was placed in the largest gap of the test AUCs (0.82 to 0.895) and of the validation AUCs (0.81 to 0.907). It was therefore not chosen in advance, and test results influenced it. Given the cutoff, the spread threshold was chosen on validation predictions and validation AUCs only. It is the middle, on a log scale, of the gap between the largest S of a collapsed run (1.0e-04) and the smallest S of a working run (0.071), which is 2.7e-03. The test predictions and test AUCs were used afterwards, to count the flagged runs. At this cutoff the threshold flags 4 runs out of 60, and those are the 4 with a test AUC of 0.85 or lower. No working run is flagged. D, with its threshold chosen the same way, flags the same 4. Because of the way the cutoff was chosen, this result is a check of the threshold and not an independent

test of the cutoff. The reference model did not add anything here.

The score is for collapse, not for every loss of accuracy. The Mercury runs under grouped normalization lose about 0.09 AUC and their spread stays above 0.76, so the check does not flag them. Venus on seed 3 is the hard case. Its S is 0.073, which is about ten times below a healthy run, and it still ranks missions at an AUC of 0.92. It is above the threshold and is not flagged.

Table V shows how the result depends on the cutoff. A single threshold can be set on the validation runs only at cutoffs of 0.85 and 0.90. Below 0.85 the Venus run on seed 2, with a validation AUC of 0.81, is labeled as working, but its spread (7.9e-05) lies inside the range of the collapsed runs, so no threshold separates the two labeled groups. At 0.95 fourteen runs count as collapsed, including Mercury runs that keep their spread, and the groups overlap again. The ranking is more stable than the threshold: the ROC AUC of S on the test predictions is 1.000 for every cutoff from 0.60 to 0.85 and 0.993 at 0.90. We read this as follows. The score separates runs with a near-constant output from the other runs. Whether a run with a constant output but a usable ranking, such as Venus on seed 2, is called collapsed depends on the AUC cutoff.

To test the threshold on seeds it was not chosen on, we set it on the validation predictions of four seeds and applied it to the test predictions of the fifth, for each seed in turn. At a cutoff of 0.85 this gives 4 collapsed runs, 0 missed and 0 false alarms over the 60 runs. At 0.90 there are 5 collapsed runs, 1 missed (a Mercury run that keeps its spread) and 0 false alarms. At the lower cutoffs no threshold could be set on some of the seeds, for the reason given above. These seeds redraw the split and the initialization, but they use the same four targets, so this is not a test on new targets.

The statistic is an ordinary output variance, and we do not claim that it is the best summary. Table V also gives the ROC AUC of three other label-free summaries of the same predictions: the interquartile range, the range, and the mean binary entropy. Up to a cutoff of 0.85 all of them separate the collapsed runs well (0.98 to 1.00). At 0.90 the spread score is the highest (0.993, against 0.989 for the entropy, 0.978 for the range and 0.945 for the interquartile range), and at 0.95 the range and the entropy are higher than it. The differences are small and rest on four collapsed runs from one target. What these results support is that a summary of the predictions per group detects a failure that the tested XGBoost baseline does not detect and that a pooled score shows only in part. They do not support a claim that this score is better than other output summaries.

A pooled score shows the collapse only in part. For the inner group (Mercury, Venus and Mars) the AUC over the test missions of all three targets is 0.9995 per timestep and 0.875 under grouped normalization, averaged over the five seeds. That is a visible drop. It is, however, 0.19 above the AUC of Venus alone (0.687), and above the mean of the three per target AUCs (0.845), because a model that is constant within a group still ranks the groups against each other. The pooled value also does not say which target failed. The spread is computed per target, and it is 3.1e-05 to 9.7e-05 for the four collapsed Venus runs, against 0.7 or more for Mercury and Mars under grouped normalization.

The spread is computed over a batch of predictions, so it depends on the batch size and on how the probabilities are calibrated. We resampled the saved test predictions at random. Batches of 30 or more missions flagged every collapsed run and no working run. Batches of 10 flagged 1.1% of the working batches and batches of 5 flagged 7.4%. These batches contain both outcomes, because they were drawn at random from the test missions. A batch that holds missions of one outcome only, or from one narrow region of the input space, was not tested, and a small spread would be expected there. We also rescaled the predicted log-odds by a temperature from 0.25 to 4. The smallest spread of a working run then moved between 0.28 and 0.018, and the threshold of 2.7e-03 still separated the 4 collapsed runs from the 56 working runs. This is a monotone change of the outputs of one model family. A model with different calibration, architecture, label prevalence or batch size needs its own threshold.

The signal ratio of Section IV can be measured before any training. Every collapsed run has it at 0.0022 or 0.0023. Five runs are below 0.01, which are the 4 collapsed runs and Venus on seed 3, and no run at 0.0243 or above collapsed. So it works as an early warning, but it cannot say which of those runs will collapse. For that, the spread after training is needed.

VII. A SECOND DATASET

To see what happens outside our simulator we used the NASA C-MAPSS turbofan data [41], which is a standard public set for remaining useful life work [42]. It has run to failure sensor records for fleets of engines. FD001 has one operating condition. FD002 and FD004 have six. The six conditions play the role the targets play in our data: the sensors sit at very different levels in each one, and the slow drift that shows an engine is wearing out is small next to that gap.

The task is to look at the last 30 cycles of an engine and say whether it will fail within 30 cycles. The split is by engine, 70/15/15, so no engine is in two sets. We scale the 14 sensors that change over an engine's life in two ways. Pooled uses one median and IQR for every row. Per group uses one for each operating condition. The model is the Transformer of

TABLE V

Dependence on the AUC cutoff that defines a collapsed run. Collapsed runs are counted on validation and on test. The threshold is set on validation only, and 'none' means that the labeled groups overlap on validation. Missed runs and false alarms are counted on test. The last four columns give the ROC AUC, on test, of four label-free summaries of the predictions.

Cutoff	Collapsed (val/test)	Threshold	Missed	False alarms	S	IQR	Range	Entropy
0.60	2/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.70	3/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.80	3/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.85	4/4	2.7e-03	0	0	1.000	1.000	1.000	1.000
0.90	4/5	2.7e-03	1	0	0.993	0.945	0.978	0.989
0.95	14/14	none	n/a	n/a	0.879	0.787	0.984	0.972

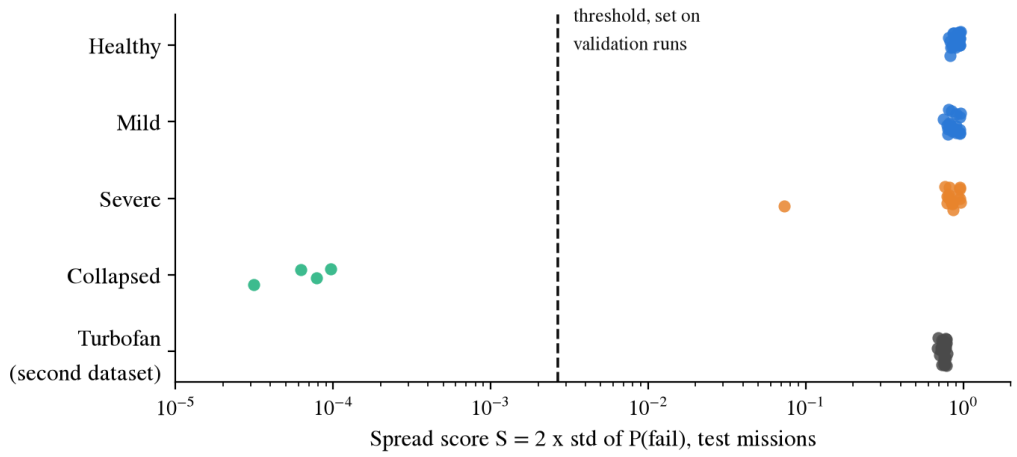


Fig. 6. The spread score of every run on the test missions. The dashed line is the threshold chosen on the validation runs. The turbofan runs of Section VII are shown for comparison, and none is flagged.

TABLE VI

The 60 runs by state. S is the spread score on the test missions, and AUC is the median on the test missions. The spread of the collapsed runs is at least 750 times smaller than that of any other run.

State	Runs	S (smallest to largest)	Median AUC
Healthy	20	0.807 to 0.963	0.9996
Mild	20	0.748 to 0.962	0.9955
Severe	16	0.073 to 0.961	0.9430
Collapsed	4	3.15e-05 to 9.69e-05	0.5912

TABLE VII

Turbofan data, 5 seeds. 'signal' is the mean within group standard deviation of the scaled features. AUC is on test engines, mean and standard deviation. S is the spread score of the Transformer.

Subset	Scaling	signal	Transformer AUC	Tree AUC	S
FD001	pooled	0.784	0.9916 ± 0.0017	0.9953	0.760
FD001	per-group	0.784	0.9916 ± 0.0017	0.9953	0.760
FD002	pooled	0.083	0.9898 ± 0.0014	0.9862	0.767
FD002	per-group	0.802	0.9855 ± 0.0025	0.9925	0.755
FD004	pooled	0.097	0.9910 ± 0.0010	0.9890	0.734
FD004	per-group	0.859	0.9881 ± 0.0044	0.9933	0.731

Section III, with the same optimizer, trained for 20 epochs on 5 seeds, and the tree is XGBoost on the same scaled window. FD001 has a single condition, so the two scalings are the same thing there. It is our control.

Pooling shrinks the signal about nine times, from 0.802 to 0.083 on FD002 and from 0.859 to 0.097 on FD004. The model does not break (Table VII). On both subsets the pooled Transformer is slightly ahead of the per group one, the spread of its predictions is the same, and the tree is fine either way. The check of Section VI, with the threshold it already had, flags none of the 30 runs.

This agrees with Section IV. The signal shrinking is not enough to break a model. In our data, Mercury and Jupiter lose most of their signal under grouped scaling and keep working. Here the drift is still there after pooling, and the model finds it. It also shows that the check does not raise an alarm on a second kind of data.

VIII. WHERE THE SCREEN BELONGS

If it has been thoroughly done on the sequence model, the real answer is that it should not be used. The six feature classifier, which is evaluated prior to

TABLE VIII

Screening economics. Compute is charged in propagation days, which span about a 100x range across the seven targets. Thresholds are fitted on validation and reported on the untouched test split, so the recall column is measured out of sample rather than achieved by construction.

Screen	Compute saved	Good missions lost	Failure recall
T0: six launch parameters, before propagating	64.5%	0.76%	99.06%
T40: telemetry Transformer at 40%	38.3%	0.21%	98.87%
Cascade: T0 where confident, else T40	65.0%	1.62%	99.64%

any propagation whatsoever, saves 64.5% of compute against the telemetry model's 38.3%, at a similar cost in good missions destroyed. There is also a leveling off of accuracy between 10% and 40% observed. The problem is not a modeling fault. The simulator is deterministic, the outcome is a fixed function of the injection offsets, and the trajectory is an integral of the offsets. It cannot transport information that the parameters do not have.

This is not to imply that the task is trivial. Logistic regression on the same 6 features achieves 0.4824 to 0.5410 AUC on all of the targets, which is chance, so the mapping between the parameters and the outcome is very nonlinear and requires a learned model. A temporal one is not really necessary. The sequential screen is only worth applying if the outcome is not known at injection: mid flight stochasticity, unmodeled dynamics, sensor noise, or campaigns where the launch parameters are not known. That is exactly the setting reinforcement learning trajectory work needs when it certifies policies through dispersed Monte Carlo campaigns [30].

For the T0 screen, an additional five-split analysis (seeds 0 to 4) gives compute saved 64.50% ± 0.61 percentage points, good missions lost 1.24% ± 0.24 percentage points, and failure recall 98.99% ± 0.18 percentage points (mean ± standard deviation across five splits). The T40 and cascade rows in Table VIII remain single seed-42 estimates and are not presented with an interval. The multi-seed T0 result narrows sampling uncertainty for that screen, but five correlated resplits of one synthetic corpus do not measure cross-campaign or cross-domain uncertainty.

IX. LIMITATIONS

- Seeds. Replication is limited to 5 seeds. The observed 4/5 recurrence has a wide 95% Wilson interval (0.376 to 0.964), and five seeds exclude neither an unknown controlling factor nor substantial uncertainty.
- Threshold. The AUC cutoff that defines a collapsed run (0.85) was set after the validation and test AUCs had been seen. The spread threshold (2.7e-03) was chosen on validation predictions only, so the test result is a check of that threshold and not an independent test of the cutoff. A single threshold can be set only for cutoffs of 0.85 and 0.90. All 4 collapsed runs are Venus, so one kind of collapse has been tested. The threshold is not expected to transfer unchanged to another dataset, model architecture, output calibration, target group, prevalence or batch size, and batches with one outcome class were not tested. A new setting needs labeled development runs to calibrate it and an untouched test to evaluate it. The spread statistic itself needs no labels at inference time.
- Datasets. The collapse is shown on the GMAT corpus. The turbofan data show the same signal loss without a collapse and without an alarm. That supports the absence of false alarms there, but it does not show that the threshold carries over to other domains.
- Four targets in the ablation. 1 of 4 targets collapses, and all collapsed runs are Venus. Other target combinations and group definitions are needed to estimate how much group composition matters. The four targets agree only that the tested XGBoost AUC is nearly invariant across these normalization settings.
- Architecture scope. An LSTM control on the ordinary, correctly preprocessed screening task reaches test AUC 0.9928 versus 0.9958 for the Transformer, showing that the base task is not unique to one sequence architecture. The LSTM was not run under grouped normalization, so whether this collapse generalizes beyond the Transformer configuration remains open.
- Severity range. The check flags near-constant output. It does not flag a model that loses accuracy and keeps its spread, such as Mercury. We have not built graded signal compression sweeps, so false alarms and misses over a broader range of conditions are not reported.
- Synthetic and deterministic. There are no execution errors, no unmodeled accelerations, no sensor noise and all missions of a target have a common time base. This common time base is what makes standardization per timestep possible at all and it is not a property of a real campaign, but of the generator.
- Real mission data. Public satellite telemetry benchmarks exist [43], [44] and are the natural next place to try the check.

X. CONCLUSION

When one scaler is pooled across groups with different scales, the tested Transformer can become a per-group constant while the tested scale-invariant

XGBoost baseline still finds the same arrays separable. None of the pieces here are exotic: the defect enters through preprocessing and survives pooled validation. The exact conclusion is not that trees cannot diagnose preprocessing, but that a diagnostic invariant to this transformation cannot answer whether a magnitude-sensitive network can use the compressed signal.

We do not claim the fix, normalization per group, to be new. The detection lesson is to monitor group-level output behavior in addition to pooled scores and baselines. Computing prediction spread requires no inference-time labels, but turning it into a go/no-go alarm requires version-specific calibration and independent labeled evaluation during development. Our experiment demonstrates a conspicuous signature and a failure mechanism. On a second dataset, where pooling shrank the signal and the model survived, the same check raised no alarm. The mission-assurance workflow, threshold transfer study, broader severity sweep, and fall-back policy are operational implications still to be validated.

XI. CODE AND DATA AVAILABILITY

All the scripts for the experiments, the JSON artifacts that each number is read from, the code for the figures and the source of this document are at https://github.com/Haise-727/gmat-pred.

The JSON creates the tables and figures in this manuscript and not the other way round. No number was typed into the manuscript by hand.

ACKNOWLEDGMENT

AI use disclosure. During the preparation of this work the authors used Claude (Anthropic) to write and debug the experiment code, to draft the text of all sections, and to check the references and the numbers against the result files. The involvement of the tool was large in the code and in the first drafts of the text. The authors reviewed, edited and verified the content and take full responsibility for it.

REFERENCES

[1] C. Nickel, D. J. Dichmann, R. Lebois, S. Lutz, and J. J. K. Parker
Monte carlo analysis as a trajectory design driver for the TESS mission
In AIAA/AAS Astrodynamics Specialist Conference, 2016.

[2] M. Massari, P. Di Lizia, and M. Rasotto
Nonlinear uncertainty propagation in astrodynamics using differential algebra and graphics processing units
Journal of Aerospace Information Systems, vol. 14, no. 9, pp. 493–503, 2017.

[3] B. Jia and M. Xin
Data-driven surrogate model and admissible-region-based orbital uncertainty propagation
Journal of Aerospace Information Systems, vol. 19, no. 12, 2022.

[4] T. Chen and C. Guestrin
XGBoost: A scalable tree boosting system
In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[5] L. Grinsztajn, E. Oyallon, and G. Varoquaux
Why do tree-based models still outperform deep learning on tabular data? In Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 507–520.

[6] R. Shwartz-Ziv and A. Armon
Tabular data: Deep learning is not all you need
Information Fusion, vol. 81, pp. 84–90, 2022.

[7] S. Ioffe and C. Szegedy
Batch normalization: Accelerating deep network training by reducing internal covariate shift
In International Conference on Machine Learning (ICML), 2015, pp. 448–456.

[8] J. L. Ba, J. R. Kiros, and G. E. Hinton
Layer normalization
arXiv preprint arXiv:1607.06450, 2016.

[9] Y. Wu and K. He
Group normalization
In Proceedings of the European Conference on Computer Vision (ECCV), 2018, pp. 3–19.

[10] D. Ulyanov, A. Vedaldi, and V. Lempitsky
Instance normalization: The missing ingredient for fast stylization
arXiv preprint arXiv:1607.08022, 2016.

[11] L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu, and L. Shao
Normalization techniques in training DNNs: Methodology, analysis and application
IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 8, pp. 10 173–10 196, 2023.

[12] J. Sola and J. Sevilla
Importance of input data normalization for the application of neural networks to complex industrial problems
IEEE Transactions on Nuclear Science, vol. 44, no. 3, pp. 1464–1468, 1997.

[13] J. Deng, X. Chen, R. Jiang, X. Song, and I. W. Tsang
ST-Norm: Spatial and temporal normalization for multi-variate time series forecasting
In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021, pp. 269–278.

[14] N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis
Deep adaptive input normalization for time series forecasting
IEEE Transactions on Neural Networks and Learning Systems, vol. 31, pp. 3760–3765, 2020.

[15] R. Geirhos et al.
Shortcut learning in deep neural networks
Nature Machine Intelligence, vol. 2, no. 11, pp. 665–673, 2020.

[16] H. Shah, K. Tamuly, A. Raghunathan, P. Jain, and P. Netrapalli
The pitfalls of simplicity bias in neural networks
In Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9573–9585.

[17] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K. Müller
Unmasking clever hans predictors and assessing what machines really learn
Nature Communications, vol. 10, no. 1, p. 1096, 2019.

[18] D. K. Chaudhary, S. Srivastava, and V. Kumar
A review on hidden debts in machine learning systems

In Second International Conference on Green Computing and Internet of Things (ICGCIoT), 2018.

[19] G. Ke et al.
LightGBM: A highly efficient gradient boosting decision tree
In Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 3146–3154.

[20] J. Pasaribu, N. Yudistira, and W. F. Mahmudy
Tabular data classification and regression: XGBoost or deep learning with retrieval-augmented generation
IEEE Access, vol. 12, pp. 191 719–191 732, 2024.

[21] V. Papyan, X. Y. Han, and D. L. Donoho
Prevalence of neural collapse during the terminal phase of deep learning training
Proceedings of the National Academy of Sciences, vol. 117, no. 40, pp. 24 652–24 663, 2020.

[22] Y. Ovadia et al.
Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift
In Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[23] S. Rabanser, S. Günnemann, and Z. C. Lipton
Failing loudly: An empirical study of methods for detecting dataset shift
In Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[24] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley
The ML test score: A rubric for ML production readiness and technical debt reduction
In 2017 IEEE International Conference on Big Data, 2017, pp. 1123–1132.

[25] D. Izzo, M. Märtens, and B. Pan
A survey on artificial intelligence trends in spacecraft guidance dynamics and control
Astrodynamics, vol. 3, no. 4, pp. 287–299, 2019.

[26] D. Izzo, E. Blazquez, R. Ferede, S. Origer, C. De Wagter, and G. C. H. E. de Croon
Optimality principles in spacecraft neural guidance and control
Science Robotics, vol. 9, no. 91, p. eadi6421, 2024.

[27] C. Sánchez-Sánchez and D. Izzo
Real-time optimal control via deep neural networks: Study on landing problems
Journal of Guidance, Control, and Dynamics, vol. 41, no. 5, pp. 1122–1135, 2018.

[28] H. Li, S. Chen, D. Izzo, and H. Baoyin
Deep networks as approximators of optimal low-thrust and multi-impulse cost in multitarget missions
Acta Astronautica, vol. 166, pp. 469–481, 2020.

[29] D. Izzo, C. Sprague, and D. Tailor
Machine learning and evolutionary techniques in interplanetary trajectory design
In Optimization in Space Engineering, 2019.

[30] A. Zavoli and L. Federici
Reinforcement learning for robust trajectory design of interplanetary missions
Journal of Guidance, Control, and Dynamics, vol. 44, no. 8, pp. 1440–1453, 2021.

[31] B. Li, J. Huang, Y. Feng, F. Wang, and J. Sang
A machine learning-based approach for improved orbit predictions of LEO space debris with sparse tracking data from a single station
IEEE Transactions on Aerospace and Electronic Systems, vol. 56, no. 6, pp. 4253–4268, 2020.

[32] X. Zhou, D. Qiao, and X. Li
Neural network-based method for orbit uncertainty propagation and estimation
IEEE Transactions on Aerospace and Electronic Systems, vol. 60, no. 1, pp. 1176–1193, 2024.

[33] S. K. Ibrahim, A. Ahmed, M. A. E. Zeidan, and I. E. Ziedan
Machine learning methods for spacecraft telemetry mining
IEEE Transactions on Aerospace and Electronic Systems, vol. 55, no. 4, pp. 1816–1827, 2019.

[34] K. R. Naik, A. Holmgren, and J. Kenworthy
Using machine learning to automatically detect anomalous spacecraft behavior from telemetry data
In IEEE Aerospace Conference, 2020.

[35] S. Jiang, Y. Jiang, Y. Wang, X. Zhang, and Z. Zhang
Anomaly detection in spacecraft telemetry data based on transformer-LSTM
In International Conference on Intelligent Communication and Networking (ICN), 2023.

[36] T. Uriot et al.
Spacecraft collision avoidance challenge: design and results of a machine learning competition
arXiv preprint arXiv:2008.03069, 2020.

[37] G. Acciarini et al.
Kessler: A machine learning library for spacecraft collision avoidance
In 8th European Conference on Space Debris, 2021.

[38] E. Stevenson, V. Rodríguez-Fernández, H. Urrutxua, and D. Camacho
Benchmarking deep learning approaches for all-vs-all conjunction screening
Advances in Space Research, vol. 72, no. 7, pp. 2660–2675, 2023.

[39] J. Yin, F. Wang, and M. Shankar
Strategies for integrating deep learning surrogate models with HPC simulation applications
In IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2022.

[40] S. P. Hughes, R. H. Qureshi, D. S. Cooley, and J. J. K. Parker
Verification and validation of the general mission analysis tool (GMAT)
In AIAA/AAS Astrodynamics Specialist Conference, 2014.

[41] A. Saxena, K. Goebel, D. Simon, and N. Eklund
Damage propagation modeling for aircraft engine run-to-failure simulation
In International Conference on Prognostics and Health Management, 2008, pp. 1–9.

[42] X. Li, Q. Ding, and J.-Q. Sun
Remaining useful life estimation in prognostics using deep convolution neural networks
Reliability Engineering & System Safety, vol. 172, pp. 1–11, 2018.

[43] K. Kotowski et al.
European space agency benchmark for anomaly detection in satellite telemetry
arXiv preprint arXiv:2406.17826, 2024.

[44] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström
Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding
In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2018, pp. 387–395.

Harsha Vardan M Sakamuri is pursuing the B.Tech. degree in Computer Science and Engineering (core) at the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, India, from 2024 to 2028. Their work

centers on system administration and DevOps, and on automation using artificial intelligence and machine learning.

Rohit Michael Raj is with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, India. Their interests are artificial intelligence and machine learning, and they want to do research on problems that help industry.

Valarmathi P. is with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, India. [Position, highest degree and where it was earned.] [Research interests.] [Selected publications or years of experience.]